

Modelo End-To-End de reconocimiento automático del habla en español

Trabajo Final de Redes Neuronales

Denise Giselle Gayet^{†1}

[†]*Departamento de Electrónica, Facultad de Ingeniería de la Universidad de Buenos Aires
Paseo Colón 850, Ciudad Autónoma de Buenos Aires, Argentina*

¹*dgayet@fi.uba.ar*

Resumen— El reconocimiento automático del habla es el procesamiento de la voz humana con el objetivo de transcribir una secuencia verbal en una secuencia de texto. El *Deep-Learning* ha revolucionado la manera de modelar los sistemas de reconocimiento de habla, introduciendo modelos *end-to-end* mediante el cual se pudo simplificar el proceso de modelización y decodificación, optimizando los parámetros del modelo con una sola función objetivo. En este trabajo se presentan varios modelos de ASR (*Automatic Speech Recognition*) cuya arquitectura está basada en redes convolucionales y recurrentes, y se realiza una comparación de rendimiento entre los diversos modelos. Los resultados sugieren que los modelos basados en redes neuronales tienen un gran potencial para resolver este tipo de problemas y que la profundidad de dichas redes juegan un rol fundamental a la hora de elegir un modelo óptimo.

Palabras clave— automatic speech recognition; deep-learning; speech-to-text; recurrent neural networks; convolutional neural networks; connectionist temporal classification

I. INTRODUCCIÓN

El reconocimiento del habla es una rama del procesamiento del habla que busca transformar las señales de audio en texto escrito. Para lograr esto, comúnmente se utiliza un enfoque probabilístico en el que la secuencia verbal se corresponde con una secuencia de texto con cierta probabilidad. Luego, para elegir la secuencia de texto que mejor representa la secuencia verbal, se busca maximizar la verosimilitud *a posteriori*:

$$\hat{S} = \arg \max_{s \in S} \mathbb{P}(s|X) \quad (1)$$

donde X es la secuencia verbal, s es una secuencia de texto posible, y $\hat{S}$ es la secuencia de texto con mayor probabilidad.

El esquema tradicional de reconocimiento del habla basado en fonemas se compone de 3 módulos fundamentales:

- El modelo acústico:
 - modela la señal acústica como secuencia de fonemas.

- alinea la secuencia acústica con la de fonemas utilizando Cadenas Ocultas de Markov, dado que naturalmente tendrán distinta longitud.
- produce una salida que es una posible secuencia de fonemas o la probabilidad de emisión

- El léxico:

- convierte la secuencia de fonemas en secuencia de caracteres de palabras utilizando un diccionario de pronunciaciones
- puede generar varias posibles secuencias de palabras

- El modelo de lenguaje:

- desambigua las posibles secuencias de palabras en la más probable

Este tipo de modelos se mantuvo como estado del arte durante años. Sin embargo, la complejidad de los modelos HMM aumenta con el tamaño del vocabulario y la variabilidad del habla, lo que llevó a problemas de escalabilidad. Además, la optimización de cada módulo se realiza por separado, pero el *decoding* se debe hacer integrando todos los módulos.

Con el auge del *Deep-Learning* se comenzaron a desarrollar modelos de reconocimiento del habla *end-to-end*, los cuales implementan una red neuronal profunda que mapea directamente la señal de voz a una secuencia de caracteres o palabras. Esta visión simplifica considerablemente el proceso de modelización y decodificación y optimiza el modelo completo con una sola función de pérdida.

En este trabajo, se propuso una arquitectura compuesta por redes convolucionales (para la extracción de *features* de las señales), redes recurrentes (para la modelización de la dependencia temporal) y perceptrones multicapa (para la clasificación de cada muestra temporal en un carácter) que toma como entrada el espectrograma Mel de una señal de audio y predice una secuencia de caracteres de texto. Se entrenaron múltiples redes de distinta profundidad y se comparó su rendimiento mediante el CER (*Character Error Rate*).

II. SET DE DATOS

Para este estudio se utilizó el set de datos Latino 40, el cual contiene señales de audio en español de

entre 3 y 4 segundos de duración con sus respectivas transcripciones. Se utilizaron 3994 señales para entrenar las redes neuronales, las cuales constituyen aproximadamente 5 horas de entrenamiento. Se utilizó un set de 500 señales para realizar la validación.

III. PRE-PROCESAMIENTO

Previo al entrenamiento de la red, se realizó sobre el set de datos un pre-procesamiento, para extraer características útiles de las señales de audio. Además, en cada *epoch* de entrenamiento, se utilizó una técnica de *data augmentation* para diversificar el set de datos.

III-A. Espectrograma MEL

El espectrograma de una señal es una representación visual del contenido en frecuencia de una señal en función del tiempo. Es decir, muestra cómo se distribuyen las frecuencias de una señal de audio a lo largo del tiempo basándose en la Transformada de Fourier. Sin embargo, el espectrograma tradicional representa la escala de frecuencias de manera lineal, a diferencia del oído humano, el cual percibe la distancia entre frecuencias de manera logarítmica. Es decir, el humano tiene mayor capacidad de detectar variaciones en frecuencias bajas que en altas.

Por esta razón, se usa una escala de frecuencias llamada Escala Mel (proveniente de melody), que mide la distancia entre frecuencias asemejándose a la percepción humana. Esto se realiza con un banco de filtros triangulares de ancho variable y creciente que agrupa frecuencias en un mismo *bin*, como se observa en la figura 1, donde se utilizan 10 *bins* (el cual en general es un parámetro a elección).

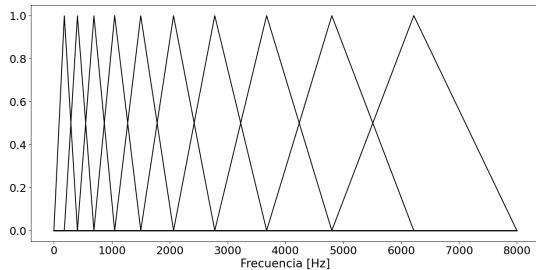


Figura 1: Banco de filtros Mel

Luego, el Espectrograma Mel se corresponde con el espectrograma tradicional de una señal en la escala Mel. Este método se utiliza por sobre el tradicional en reconocimiento de habla no solo porque posee mayor capacidad para modelar las propiedades del habla, sino también porque es una manera de reducir la dimensionalidad de los datos de entrada.

Para realizar el pre-procesamiento de las señales de audio del dataset se utilizaron 128 *bins*.

III-B. Data Augmentation

Data Augmentation es una técnica utilizada para aumentar artificialmente la diversidad de un set de datos con el objetivo de aumentar efectivamente el tamaño del mismo. Esta estrategia es especialmente útil en

los casos en que los datos son escasos, o en los que el modelo tiene sobre-adaptación a los datos de entrenamiento.

La técnica utilizada en este trabajo consiste en suprimir bloques aleatorios del espectrograma tanto en tiempo como en frecuencia. Se aplica sobre cada muestra y se repite aleatoriamente en cada *epoch*. En la figura 3 se muestra un ejemplo del procesamiento aplicado sobre un espectrograma Mel.

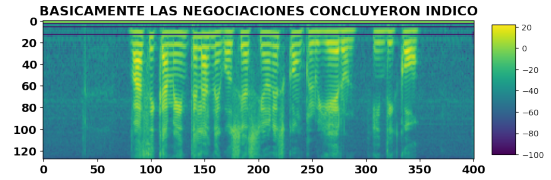


Figura 2: Espectrograma Mel original

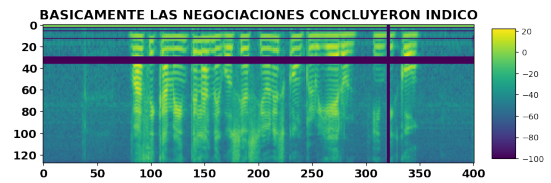


Figura 3: Espectrograma Mel procesado

Si bien es una técnica sencilla, se verificó que es efectiva en la mejora del rendimiento de sistemas de reconocimiento automático del habla [1].

Por último, para una señal de T muestras temporales, la dimensión de la entrada a la red será: $[128, T]$.

IV. ARQUITECTURA DE LA RED

En la figura 4 se encuentra un diagrama de la arquitectura de la red neuronal, basada en [2] y [3], la cual está constituida por:

- **N capas de redes convolucionales residuales:** extractores de *features*
- **Perceptrón multicapa:** aplanan el mapa de activación para alimentar a la capa recurrente
- **M capas de redes recurrentes tipo GRU Bi-direccional:** extractores de información secuencial
- **Perceptrón multicapa:** transformación de la salida en una matriz de probabilidades.

IV-A. Res-CNN

Para capturar la información espacial que guardan los espectrogramas se implementó una red convolucional, las cuales son utilizadas para reconocer patrones en imágenes, dado que son capaces de aprender características invariantes a la translación. Esto es, pueden identificar dichos patrones en distintas partes de la señal, independientemente de dónde se encontraban durante el entrenamiento.

La arquitectura utilizada para las capas convolucionales fue la siguiente:

- 1 capa con un kernel de tamaño 3×3 , *stride* de 2, y *padding* de 1; con un canal de entrada y 32 canales

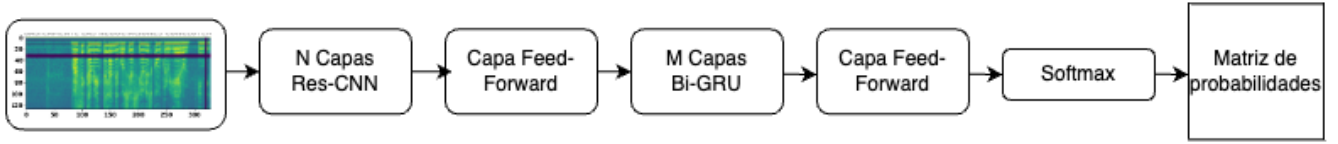


Figura 4: Arquitectura de la red

de salida

Luego, N capas de:

- 1 capa con un kernel de tamaño 3x3, *stride* de 1 y *padding* de 1; 32 canales de entrada y 32 canales de salida
- 1 capa con un kernel de tamaño 3x3, *stride* de 1 y *padding* de 1; 32 canales de entrada y 32 canales de salida
- una conexión directa en paralelo desde la entrada hasta la salida

El hiper-parámetro N se dejó como un parámetro variable y se realizaron múltiples entrenamientos variando el mismo.

Luego de pasar el espectrograma por la red convolucional, se concatenan los canales del mapa de activación (la salida). El objetivo es adecuarlo para ser la entrada de una red *feed-forward* de 1 capa que se utiliza para transformar la entrada de dimensión $[64 \cdot 32, T]$ a la dimensión necesaria para ser la entrada de la red recurrente.

IV-B. GRU Bidireccional

Las redes neuronales recurrentes son utilizadas para modelar secuencias de datos temporales, ya que pueden procesar los *features* de una señal de audio paso a paso, y realizar una predicción para cada muestra de tiempo utilizando el contexto de tiempos anteriores. La manera de capturar información a lo largo del tiempo es mediante la introducción de **estados ocultos**. El cómputo de un estado oculto en un tiempo dado se realiza mediante una combinación lineal de la entrada de la red en ese instante y del estado oculto en un instante anterior. Los pesos con los cuales se calcula dicha combinación lineal son los parámetros a aprender (los cuales son invariantes a lo largo del tiempo).

Sin embargo, las redes recurrentes tradicionales presentan dificultades para capturar dependencias a largo plazo, dado que para realizar la propagación *backwards* primero se necesita procesar toda la secuencia. Luego, al propagar el gradiente hacia el tiempo inicial, el mismo se desvanece. Más aún, a medida que se propaga hacia adelante en el tiempo, se pierde todo recuerdo de la entrada en instantes lejanos.

La solución propuesta para este problema es la implementación de celdas tipo GRU (*Gated Recurrent Unit*), que usan compuertas de actualización y reinicio, las cuales controlan el flujo de información a lo largo de la secuencia.

En la figura 5 se muestra un diagrama de una celda GRU, la cual funciona de la siguiente manera:

- en el tiempo t se tiene a disposición la información de la entrada en t y el estado oculto en $t - 1$
- *update gate*: obtiene un número entre 0 y 1 a partir de una combinación lineal entre la entrada y el estado oculto anterior, el cual da una medida de cuánta información del **estado oculto anterior** y cuánta información de la **entrada** se propaga hacia el estado oculto actual
- *reset gate*: obtiene un número entre 0 y 1 a partir de una combinación lineal entre la entrada y el estado oculto anterior, el cual da una medida de cuánta información del **estado oculto anterior** se olvida.

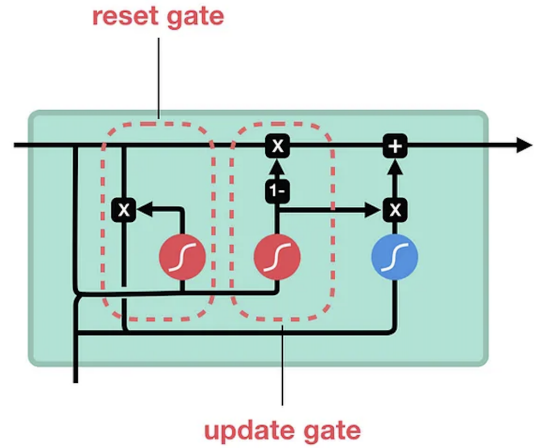


Figura 5: Celda GRU

Matemáticamente, el cálculo de la celda en un tiempo t se puede expresar como:

$$\begin{aligned} r_t &= \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{(t-1)} + b_{hr}) \\ u_t &= \sigma(W_{iu}x_t + b_{iu} + W_{hu}h_{(t-1)} + b_{hu}) \\ n_t &= \tanh(W_{in}x_t + b_{in} + r_t \cdot (W_{hn}h_{(t-1)} + b_{hn})) \\ h_t &= (1 - u_t) \cdot n_t + u_t \cdot h_{(t-1)} \end{aligned} \quad (2)$$

donde r_t representa el *reset gate* y u_t el *update gate*; W_{xy} representan las matrices de pesos y b_{xy} los *biases*; $\sigma()$ representa la función sigmoidea y $\tanh()$ la función de activación (tangente hiperbólica). Por último x_t representa la entrada en el tiempo t y $h_{(t-1)}$ el estado oculto en $t - 1$.

Algunas observaciones:

- Como se dijo anteriormente, los parámetros de la red (W_{xy} y b_{xy}) no varían a lo largo del tiempo, lo cual permite capturar información de manera secuencial.
- Se puede aumentar la profundidad de la red apilando celdas (en el mismo instante) con distintos parámetros.

A la cantidad de celdas apiladas se llama cantidad de capas.

- También es un parámetro a aprender el estado oculto en el tiempo inicial (-1).
- La dimensión del estado oculto h es un hiperparámetro a elegir al definir la red.

Luego, se pueden desglosar las celdas a lo largo del tiempo para formar una red profunda, como se observa en la figura 6:

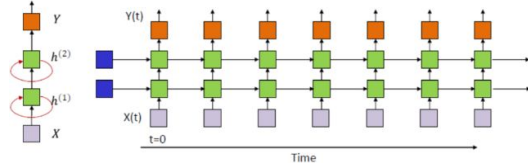


Figura 6: Diagrama de una RNN

Por último, en este trabajo se utilizaron redes bidireccionales porque no sólo es útil el contexto de tiempos pasados, sino también el contexto provisto por muestras futuras. Para implementar una red bidireccional se utiliza una red *forward*, que procesa la secuencia desde el tiempo inicial hasta el final, y una red *backward* que la procesa desde tiempo final hacia el inicial. Luego, se concatenan las salidas de cada red.

La arquitectura utilizada para la capa recurrente fue la siguiente:

- M-1 capas GRU bidireccionales, con estado oculto de dimensión 512
- 1 capa GRU bidireccional, con estado oculto de dimensión 1024

IV-C. Funciones de Activación

La función de activación utilizada en la red convolucional y las redes *feed-forward* fue la GELU(x), la cual es la función de probabilidad acumulada de una distribución normal estándar.

En la figura 7 se encuentra el gráfico la función de activación. Como se puede observar, es una función muy similar a la RELU(x). Sin embargo, tiene la ventaja de ser diferenciable en todo su dominio.

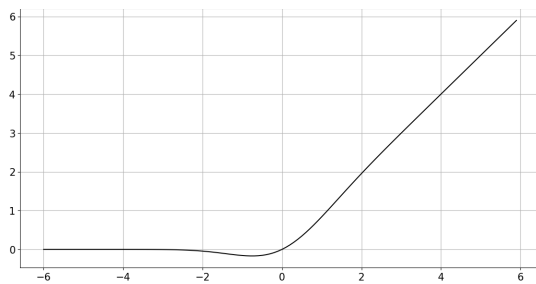


Figura 7: GELU(x)

IV-D. Clasificador

En ASR, para una señal acústica de voz se desea encontrar la secuencia de palabras emitidas. En un sistema basado en caracteres, para cada muestra temporal, se predice

el carácter emitido dentro de un conjunto de caracteres que conforman el abecedario del idioma en cuestión.

La etapa de clasificación de la red, es un perceptrón multicapa que para cada muestra, predice la probabilidad de emitir cada carácter del abecedario.

La arquitectura del perceptrón utilizado fue la siguiente:

- 1 capa con dimensión de entrada 1024 y dimensión de salida 512, con función de activación GELU(x).
- 1 capa con dimensión de entrada 512 y dimensión de salida 28 (el tamaño del abecedario), con función de activación softmax.

Luego, para una señal de audio de T muestras, la salida es una matriz como la de la figura 8:

	0				T
A	0.3	0.1	0.2	...	0.2
.	...				
.					
Z	0.5	0.01	0.1	...	0.2

Figura 8: Matriz de probabilidades

V. TÉCNICAS DE REGULARIZACIÓN

Las técnicas de regularización son estrategias diseñadas para reducir el *overfitting* de un modelo y aumentar su capacidad de generalización. En este trabajo, además de *Data Augmentation*, se utilizaron otras dos técnicas: *Layer Normalization* y *Dropout*.

V-A. Layer Normalization

Esta técnica busca estandarizar las activaciones de las neuronas en una capa específica de la red. A diferencia de *Batch Normalization*, que normaliza las activaciones a lo largo del *mini-batch*, se normaliza a lo largo de las dimensión de los *features*. Utilizandola, se puede estabilizar el aprendizaje y acelerar la convergencia.

Este método es recomendable especialmente cuando el tamaño del *mini-batch* no es lo suficientemente grande como para hacer la estandarización a lo largo de dicha dimensión.

La expresión matemática es la siguiente:

$$x_{norm} = \frac{x - E[x]}{\sqrt{V[x]}} \cdot \gamma + \beta \quad (3)$$

donde γ y β son parámetros a aprender.

Esta técnica se aplicó a la salida de cada capa de red convolucional y/o *feed-forward* y previo a la función de activación.

V-B. Dropout

Este método consiste en apagar cada neurona con una probabilidad α durante el entrenamiento, con el objetivo de reducir la co-adaptación entre neuronas, evitando que dependan demasiado de otras neuronas específicas. El resultado es una disminución en el *overfitting* del modelo.

En este trabajo, para todas las capas de neuronas se utilizó un $\alpha = 0.1$.

VI. ENTRENAMIENTO

El entrenamiento de las redes neuronales de aprendizaje supervisado se basa en un algoritmo de *backpropagation*, el cual consiste en realizar una predicción para un set de entrada, y compararlo con sus etiquetas asociadas mediante alguna función de pérdida. Luego, se calcula el gradiente de dicha función con respecto a los parámetros de la red, propagándolo capa a capa. Mediante un método de optimización se actualizan dichos parámetros y el proceso se repite hasta llegar a la convergencia.

En este trabajo se utilizó como función de pérdida el algoritmo CTC (*Connectionist Temporal Classification*) [4], y como optimizador AdamW.

VI-A. Connectionist Temporal Classification

Un problema que surge en reconocimiento de habla es el hecho de que la secuencia verbal no tiene una correspondencia uno a uno con su secuencia de texto y a priori no se conoce la alineación entre ambas. A su vez, la secuencia de texto predicha tiene la misma longitud de la secuencia verbal, con lo cual, ese necesario realizar algún tipo de alineamiento antes de aplicar la función de pérdida que compara la predicción con la etiqueta. CTC es una manera de sortear el problema de no conocer el alineamiento entre la entrada y la salida.

Considerando una secuencia de entrada $X = \{x_1, x_2, x_3, \dots, x_T\}$ y una secuencia de salida $Y = \{y_1, y_2, y_3, \dots, y_N\}$, como el ejemplo que se muestra en la figura 9, el algoritmo toma la secuencia predicha o alineamiento, y luego fusiona (colapsa) los caracteres repetidos para llegar a una secuencia de salida.

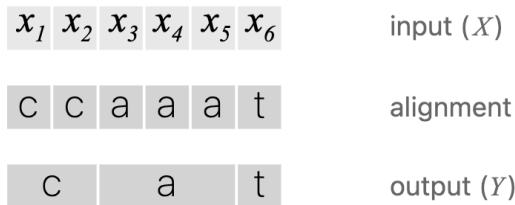


Figura 9: Ejemplo de funcionamiento de CTC

Este abordaje tiene 2 problemas fundamentales.

- No siempre tiene sentido forzar la emisión de un carácter para una entrada, dado que hay momentos de silencio.
- No se puede predecir secuencias con caracteres adyacentes repetidos. Por ejemplo: al colapsar la palabra "hello", se obtendrá "helo".

Ambos problemas se pueden resolver introduciendo al abecedario en cuestión un nuevo carácter, llamado *blank* y simbolizado mediante ϵ . Este carácter no se corresponde con nada y al final será removido de la secuencia de salida.

El ejemplo de la figura 10 es una muestra de cómo se pueden obtener secuencias con caracteres repetidos.

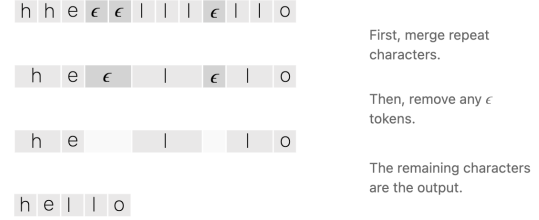


Figura 10: Ejemplo de funcionamiento de ϵ

Retomando el objetivo, el algoritmo CTC computa la probabilidad de obtener la secuencia de salida Y dada la entrada X : $P(Y|X)$. Esto lo realiza sumando sobre todas los posibles alineamientos entre ambas secuencias. Es decir, sobre todos los alineamientos *válidos*.

El algoritmo considera como alineamiento válido a todo alineamiento de la misma longitud que la entrada X , que al colapsarse (al remover caracteres repetidos y ϵ) se condice con la secuencia de salida Y . En la figura 11 se pueden encontrar varios ejemplos de alineamientos válidos e inválidos.

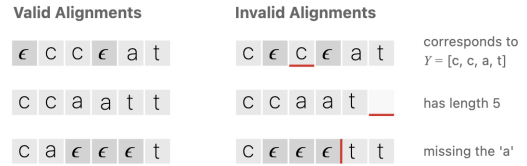


Figura 11: Ejemplo alineamientos válidos e inválidos

Luego, recordando que la salida de la red neuronal es una matriz de probabilidades de caracteres en cada muestra temporal, se pueden obtener múltiples alineaciones válidas a partir de dicha matriz. Entonces, la probabilidad de obtener una secuencia de salida se puede calcular de la siguiente manera:

$$\mathbb{P}(Y|X) = \sum_{A \in A_{X,Y}} \prod_{t=1}^T p_t(a_t|X)$$

Para cada alineamiento válido $A = \{a_1, a_2, \dots, a_T\}$ que pertenece al conjunto de alineamientos permitidos $A_{X,Y}$, se calcula la probabilidad de obtener dicho alineamiento dada la secuencia de entrada. Esto se realiza suponiendo independencia temporal entre la probabilidad de emisión de un carácter. Luego, la probabilidad del alineamiento se puede computar como la productoria de las probabilidades marginales $p_t(a_t|X)$, que se corresponden con los elementos de la matriz de probabilidades. Finalmente, la probabilidad de la secuencia de salida dada la secuencia de entrada se calcula como la sumatoria de la probabilidad de cada

alineamiento.

Por último, para un set de entrenamiento D de tamaño M la función de *Loss* que se desea minimizar es:

$$Loss(W) = \frac{1}{M} \sum_{(X,Y) \in D} -\log(P(Y|X))$$

VI-B. AdamW

Como optimizador se utilizó AdamW, una modificación de *Stochastic Gradient Descent* que define una manera de actualizar los parámetros de acuerdo a, no solo el gradiente de la función de pérdida, sino también a su media móvil (momento de primer orden) y su varianza móvil no centrada (momento de segundo orden).

La media móvil se define como:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f_{t-1}$$

y la varianza móvil no centrada como:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \nabla f_{t-1}^2$$

La varianza se llama no centrada pues no se le resta el cuadrado de la media ($Var(x) = \langle x^2 \rangle - \langle x \rangle^2$). Esto es importante, dado que, si el gradiente no varía respecto de iteraciones anteriores, la varianza será cercana a 0 y $\langle x^2 \rangle = v(t) \approx \langle x \rangle^2$. Por lo tanto, se tendrá que $\sqrt{v_t} \approx m_t$.

Estas cantidades son utilizadas para tener una medida cómo y cuánto varía el gradiente a lo largo del tiempo, y a partir de esto modificar el paso para la actualización de los parámetros: cuando el gradiente es muy errático y tiene muchas variación (el cociente entre el momento de 1er orden y la raíz del momento de 2do es pequeño) los parámetros se ajustan con variaciones pequeñas, en cambio, cuando el gradiente es estable a lo largo del tiempo (el cociente es cercano a 1) los parámetros se ajustan según la constante de aprendizaje.

Además, AdamW introduce un *weight decay* [5] sobre la actualización de los parámetros que se basa en la idea de que las redes con pesos más pequeños tienden a tener menos *overfitting* y a generalizar mejor. Este método agrega una constante λ que reduce los pesos de manera exponencial y fuerza a la red a aprender pesos más pequeños en la medida de lo posible.

En resumen, la actualización de los pesos se rige según la siguiente fórmula:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{m_t}{\sqrt{v_t} + \zeta} + \lambda \theta_t \right)$$

donde:

- θ_t son los parámetros en el tiempo t ,
- m_t es el momento de 1er orden del gradiente,
- v_t es el momento de 2do orden,
- ζ es una constante pequeña para no dividir por 0,
- λ es el parámetro de *weight decay*,
- η es la constante de aprendizaje

Para definir la constante de aprendizaje se utilizó *One Cycle Learning Rate Scheduler* [6], un método empírico que demostró que permite entrenar las redes de manera más rápida, manteniendo su capacidad de generalización. Consiste en elegir una constante de aprendizaje que empiece siendo pequeña, luego aumentarla de manera lineal hasta un máximo, y luego reducirla nuevamente hasta el mínimo inicial de manera lineal.

VII. INFERENCIA

Una vez finalizado el entrenamiento, cuando se utiliza la red para realizar predicciones, no se le provee la secuencia de texto asociada a la secuencia verbal (muchas veces no habrá una etiqueta disponible). Por lo tanto, no se puede utilizar el algoritmo CTC para obtener una secuencia de salida colapsada.

Entonces, para decodificar las salidas a partir del mapa de probabilidades, se utilizó un método *greedy*. El mismo consiste en encontrar una secuencia recorriendo el camino de mayor probabilidad marginal. Para cada tiempo, se elige el caracter con mayor probabilidad. Luego se reduce la secuencia colapsando los caracteres repetidos y eliminando de la secuencia los caracteres ϵ .

VIII. RESULTADOS

Se entrenó un total de 7 configuraciones variando los hiper-parámetros disponibles.

En todos los casos se utilizó un tamaño de *batch* de 8, una constante de aprendizaje inicial de $1 \cdot 10^{-4}$ y 100 *epochs*.

Los parámetros que se dejaron como variables son: la cantidad de capas de las redes convolucionales (N) y la cantidad de capas de las redes recurrentes (M).

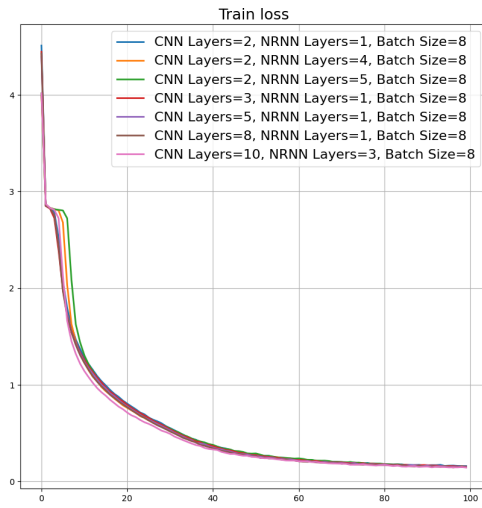
En el cuadro I se muestran las distintas configuraciones entrenadas según los parámetros N y M.

	N	M
1	2	1
2	3	1
3	5	1
4	8	1
5	2	4
6	2	5
7	10	3

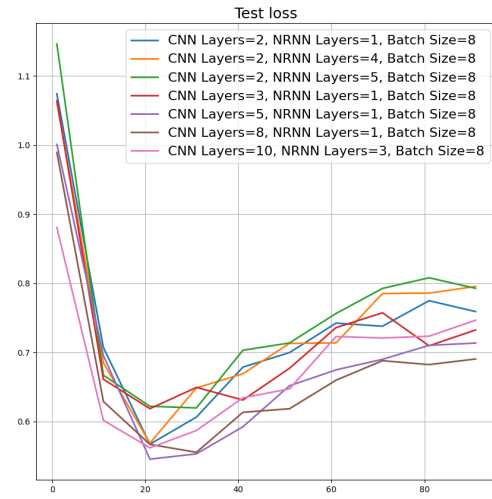
Cuadro I: Configuraciones entrenadas

Como se puede observar, la estrategia fue entrenar por un lado redes con una baja cantidad de capas recurrentes e ir aumentando la cantidad de capas convolucionales, y por otro lado, redes con una baja cantidad de capas convolucionales y aumentar la cantidad de capas recurrentes.

En la figura 12a se muestra el *Loss* de entrenamiento en función de las *epochs*, donde se puede observar que el error disminuye con cada *epoch*, lo cual significa que la red mejora su capacidad de predecir secuencias para el set de entrenamiento.



(a) Entrenamiento



(b) Validación

Figura 12: Loss

Por otro lado, en la figura 12b, gráfico que representa el *Loss* de validación, se puede apreciar claramente que el error llega a un mínimo en la *epoch* 20 y a partir de ese momento vuelve a aumentar de manera sostenida. Esto puede dar indicios de que a partir de cierto momento el modelo está sobre-adaptándose al set de entrenamiento, pues el error de entrenamiento continúa disminuyendo, pero el error de validación aumenta.

Por lo tanto, se podría concluir que se debe dejar de entrenar en el momento en que el error de validación comienza a aumentar, de lo contrario, el modelo estaría perdiendo capacidad de generalización.

Sin embargo, hay que tener en cuenta que se está observando la función pérdida, que no es la misma métrica que se utiliza para analizar el rendimiento de la red.

La métrica utilizada para analizar la *performance* es el CER (Character Error Rate). Esta medida cuenta cuántas inserciones (I), deleciones (D) y sustituciones (S) se deben realizar sobre una secuencia *target* para que sea igual a una secuencia referencia. Luego, se calcula el CER como:

$$CER = \frac{S + D + I}{N}$$

donde N es el largo de la secuencia de referencia.

La diferencia principal entre CTC y CER es que CTC mide la cantidad de caracteres equivocados promedio sobre **todas las secuencias extendidas posibles**. En cambio, el CER se mide sobre la secuencia decodificada, que además es producto de una maximización de probabilidades. Por lo tanto, hay un des-apareamiento entre la función que se puede utilizar para la minimización (CTC) y la que se utiliza para medir el rendimiento (CER).

En la figura 13 se muestra el CER de validación en función de las *epochs*. En este caso, se puede observar que a pesar de que el error de validación aumente, el CER continúa

disminuyendo, es decir, el modelo continúa aumentando su rendimiento.

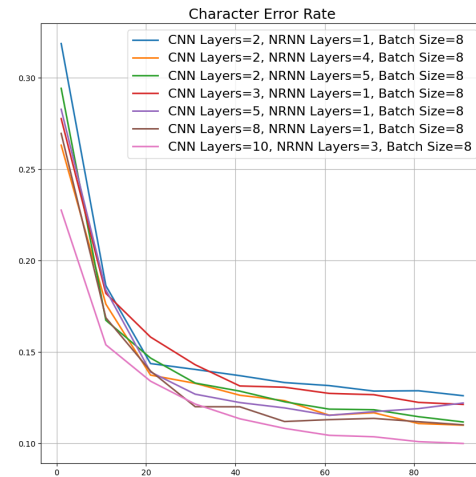


Figura 13: Test CER

Como lo que realmente mide el rendimiento de una red es el CER, se utilizó esta medida para comparar los distintos modelos entrenados.

En la figura 14 se muestra el CER Promedio de la última *epoch* para cada modelo entrenado.

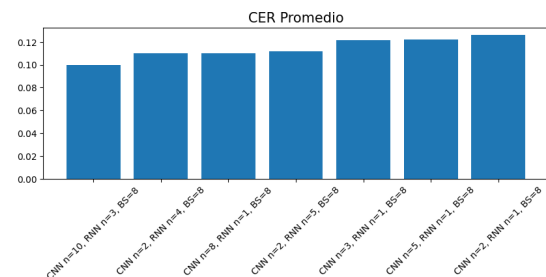


Figura 14: CER promedio

Como era de esperarse, la red de mayor profundidad es la que mejor rendimiento posee, aunque la diferencia no es

muy significativa, dado que la red con peor rendimiento tiene un porcentaje de error 12.6 %, y la de mejor rendimiento 10 %.

Además, la red que tiene más capas convolucionales (8) y una sola capa recurrente tiene un rendimiento similar a las que tienen varias capas recurrentes (4 y 5) y baja cantidad de capas convolucionales (en ambos casos, 2), siendo de los mejores rendimientos en los 3 casos. A priori, se podría pensar que es indistinto aumentar la cantidad de capas convolucionales o la cantidad de capas recurrentes.

Sin embargo, la diferencia entre las configuraciones entrenadas se puede poner en perspectiva al analizar la cantidad total de parámetros que tiene cada red, lo cual puede dar una medida de la complejidad del modelo, y por consiguiente, de la velocidad de entrenamiento.

En la figura 15 se muestra la cantidad de parámetros totales de cada red entrenada.

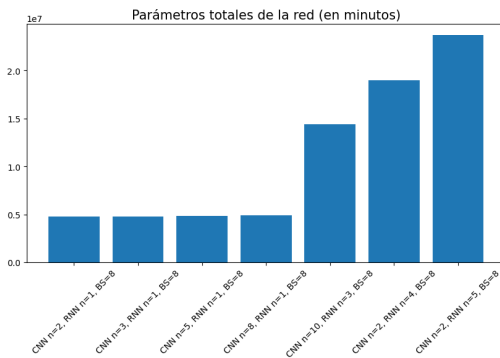


Figura 15: Cantidad de Parámetros

Como se puede observar, el modelo de 8 capas convolucionales y 1 capa recurrente, tiene una cantidad de parámetros que es el 25 % de la cantidad del modelo de 4 capas recurrentes y 2 convolucionales; y el 20 % del modelo de 5 capas recurrentes y 2 convolucionales. Más aún, el modelo con mejor rendimiento (10 capas convolucionales y 3 recurrentes) tiene menor cantidad total de parámetros que aquellos modelos con más capas recurrentes.

Por lo tanto, se puede concluir que se puede reducir la complejidad del modelo sin sacrificar el rendimiento, aumentando la cantidad de capas convolucionales de la red y manteniendo una cantidad de capas recurrentes baja. Esto puede mejorar sustancialmente el tiempo de convergencia del entrenamiento, pudiendo utilizar sets de entrenamiento de mayor tamaño sin tener tiempos de entrenamiento excesivamente altos.

IX. DISCUSIÓN Y CONCLUSIONES

En este trabajo se discutió una arquitectura de redes neuronales profundas para realizar procesamiento automático del habla de manera *end-to-end* en el lenguaje Español.

Se pudo establecer tanto la importancia de capas convolucionales para la extracción de *features*, como la

necesidad de capas recurrentes que sean capaces de capturar información que depende de un contexto y que mantiene una dependencia temporal a largo plazo.

Se obtuvieron resultados favorables (desde un 12 % hasta un 10 % de error de caracteres promedios) en todos los modelos entrenados y se puso establecer una relación entre la complejidad del modelo y las distintas configuraciones, observándose que aumentar la profundidad en las capas recurrentes presenta un gran aumento en la cantidad de parámetros a entrenar (lo que se traduce en más dificultad y mayores tiempos de entrenamiento) sin ofrecer una gran mejora en el rendimiento de la red. Este resultado puede ser especialmente útil si el hardware disponible es una limitación.

Si bien los resultados no se comparan con los obtenidos en otros modelos [2], se tiene que tener en cuenta que el *dataset* utilizado es de apenas 5 horas de entrenamiento, cuando los *datasets* disponibles en Inglés pueden ascender fácilmente a 1000 horas de entrenamiento (ej. Librispeech). Se considera que este fue el principal limitante en el entrenamiento, aún habiendo utilizado técnicas de *Data Augmentation* para aumentar el tamaño del set de datos.

Por otro lado, otro limitante a la hora de generar predicciones es el *decoder* utilizado, el cual se basa únicamente en maximizar probabilidades marginales. El rendimiento probablemente se podría mejorar utilizando un modelo de lenguaje para hacer la decodificación, o un algoritmo de búsqueda más sofisticado. Además, cabe aclarar que en este caso la función objetivo (CTC Loss) no coincidía completamente con lo que realmente se quería minimizar, el porcentaje de error de caracteres, pero que igualmente es útil para entrenar la red.

Por último, el rendimiento de una red neuronal depende mucho del *fine-tuning* de los hiper-parámetros de la red, como ser el tamaño de *batch*, las constantes de aprendizaje, la profundidad de las distintas capas, la dimensión de los estados ocultos, el tamaño de los *kernels*, la cantidad de *epochs* de entrenamiento, entre otros. Existen muchísimos hiper-parámetros que se pueden modificar para obtener un mejor modelo. Desafortunadamente, no fue posible hacer un análisis más extenso de los mismos, dado que las limitaciones de hardware impusieron una gran restricción.

REFERENCIAS

- [1] D. S. Park *et al.*, "Spec augment: A simple data augmentation method for automatic speech recognition," *Proc. Interspeech 2019*, vol. 20, pp. 2613–2617, Dec 2019.
- [2] D. Amodei *et al.*, "Deep speech 2: End-to-end speech recognition in english and mandarin," 2015.
- [3] M. Nguyen, "Building an end-to-end speech recognition model in pytorch," Dec 2021. [Online]. Available: <https://www.assemblyai.com/blog/end-to-end-speech-recognition-pytorch/>
- [4] A. Hannun, "Sequence modeling with ctc," *Distill*, 2017, <https://distill.pub/2017/ctc>.
- [5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019.
- [6] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," 2018.